TEXT CATEGORIZATION USING MACHINE LEARNING

Python NLP Laboratory — Step-by-step execution and explanation

Objective: Use Bag-of-Words and TF-IDF feature extraction to represent documents and classify them into predefined categories.

1. What is Text Categorization?

Text categorization is the process of automatically assigning a category or label to a text document. For example, “India won the cricket match” can be labelled Sports, while “Artificial Intelligence is growing rapidly” can be labelled Technology. After learning from labelled examples, a model can classify a new document.

2. How does a computer represent text?

A machine-learning algorithm works with numerical features. Therefore, text documents must first be converted into numbers. Two common methods are Bag-of-Words (BoW) and Term Frequency–Inverse Document Frequency (TF-IDF).

A. Bag-of-Words (BoW)

Bag-of-Words represents each document by counting how often each vocabulary word occurs. Consider these documents: (1) “I love machine learning”, (2) “I love deep learning”, and (3) “I study machine learning”. The vocabulary is: I, love, machine, learning, deep, study.

Word	Document 1	Document 2	Document 3
I	1	1	1
love	1	1	0
machine	1	0	1
learning	1	1	1
deep	0	1	0
study	0	0	1

Document 1 becomes [1, 1, 1, 1, 0, 0], Document 2 becomes [1, 1, 0, 1, 1, 0], and Document 3 becomes [1, 0, 1, 1, 0, 1]. These numerical lists are called feature vectors.

B. TF-IDF

TF-IDF stands for Term Frequency–Inverse Document Frequency. It gives a weight to a word based on its frequency in a document and how common or rare it is across the document collection.

TF-IDF = TF × IDF

Term Frequency (TF) measures how frequently a term occurs in a document. In this explanation, TF is calculated as the term count divided by the total number of words in that document.

TF(t,d) = Frequency of term t in document d / Total words in document d

Inverse Document Frequency (IDF) reduces the weight of terms that occur in many documents:

IDF(t) = log(N / DF(t))

N is the total number of documents, and DF(t) is the number of documents containing term t. Common words such as “is” may be less useful for distinguishing categories, while informative words such as “cricket” may help identify a sports document.

3. What is Machine Learning Classification?

After documents are converted into numerical vectors, a machine-learning algorithm learns the relationship between the vectors and their category labels. This experiment uses Multinomial Naive Bayes to classify documents into Sports and Technology.

Workflow: Input text documents → preprocessing/tokenization → feature extraction (BoW or TF-IDF) → Multinomial Naive Bayes classifier → predicted category.

4. Install the required library

Open Command Prompt or the VS Code terminal and run:

```
pip install scikit-learn
```

The program uses CountVectorizer for BoW, TfidfVectorizer for TF-IDF, train_test_split to create training and testing sets, MultinomialNB for classification, and accuracy_score/classification_report for evaluation.

5. Complete Python program

Create a Python file named **text_categorization.py** and enter the following code:

```
# TEXT CATEGORIZATION USING MACHINE LEARNING

# STEP 1: Import required libraries
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report

# STEP 2: Create sample text documents
documents = [
    "India won the cricket match",
    "The football team scored a goal",
    "The player scored a century in cricket",
    "The team won the football tournament",
    "Cricket is a popular sport",
    "The coach trained the football players",
    "The batsman hit a six in the match",
    "The goalkeeper saved the penalty",
    "The tennis player won the match",
    "The basketball team scored points",
    "Artificial intelligence is transforming technology",
    "Machine learning is used in healthcare",
    "Python is a popular programming language",
    "Deep learning is a branch of artificial intelligence",
    "Computer networks connect different devices",
    "Data science uses machine learning algorithms",
    "Cloud computing provides online services",
    "Cybersecurity protects computer systems",
    "Robotics uses artificial intelligence",
    "Software development uses programming languages"
]

# STEP 3: Define document categories
labels = [
    "Sports", "Sports", "Sports", "Sports", "Sports",
    "Sports", "Sports", "Sports", "Sports", "Sports",
    "Technology", "Technology", "Technology",
    "Technology", "Technology", "Technology",
    "Technology", "Technology", "Technology",
    "Technology"
]

# STEP 4: Divide data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    documents, labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

# STEP 5: BAG OF WORDS FEATURE EXTRACTION
print("\n===== BAG OF WORDS =====")
bow_vectorizer = CountVectorizer()
X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)

print("Vocabulary:")
print(bow_vectorizer.get_feature_names_out())
```

```
print("\nTraining Matrix:")
print(X_train_bow.toarray())

# STEP 6: Train Naive Bayes using BoW
bow_model = MultinomialNB()
bow_model.fit(X_train_bow, y_train)
bow_predictions = bow_model.predict(X_test_bow)
bow_accuracy = accuracy_score(y_test, bow_predictions)

print("\nBoW Accuracy:", bow_accuracy)
print("\nBoW Classification Report:")
print(classification_report(y_test, bow_predictions))

# STEP 7: TF-IDF FEATURE EXTRACTION
print("\n===== TF-IDF =====")
tfidf_vectorizer = TfidfVectorizer()
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

print("Vocabulary:")
print(tfidf_vectorizer.get_feature_names_out())
print("\nTF-IDF Matrix:")
print(X_train_tfidf.toarray())

# STEP 8: Train Naive Bayes using TF-IDF
tfidf_model = MultinomialNB()
tfidf_model.fit(X_train_tfidf, y_train)
tfidf_predictions = tfidf_model.predict(X_test_tfidf)
tfidf_accuracy = accuracy_score(y_test, tfidf_predictions)

print("\nTF-IDF Accuracy:", tfidf_accuracy)
print("\nTF-IDF Classification Report:")
print(classification_report(y_test, tfidf_predictions))

# STEP 9: Compare both methods
print("\n===== PERFORMANCE COMPARISON =====")
print("Bag of Words Accuracy:", bow_accuracy)
print("TF-IDF Accuracy:", tfidf_accuracy)

# STEP 10: Predict categories of new documents
new_document = [
    "Artificial intelligence helps computers learn",
    "The cricket team won the tournament"
]

print("\n===== NEW DOCUMENT PREDICTION =====")
new_bow = bow_vectorizer.transform(new_document)
bow_result = bow_model.predict(new_bow)

new_tfidf = tfidf_vectorizer.transform(new_document)
tfidf_result = tfidf_model.predict(new_tfidf)

for i, text in enumerate(new_document):
    print("\nDocument:", text)
    print("BoW Prediction:", bow_result[i])
    print("TF-IDF Prediction:", tfidf_result[i])
```

6. Explanation of the Python program

Step 1 — Import the libraries

CountVectorizer converts text into Bag-of-Words counts. TfidfVectorizer converts text into TF-IDF weights. MultinomialNB is the classifier. train_test_split divides the examples into training and testing sets. accuracy_score and classification_report evaluate predictions.

Step 2 — Create sample documents and labels

The program contains 20 labelled documents: 10 Sports documents and 10 Technology documents. Each document has a matching label. X represents the input documents and y represents their category labels.

Step 3 — Divide the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    documents, labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)
```

test_size=0.25 reserves 25% of the documents for testing and 75% for training. With 20 documents, this gives 15 training documents and 5 testing documents. random_state=42 makes the split reproducible. stratify=labels maintains the class proportions in both sets.

Step 4 — Bag-of-Words feature extraction

```
bow_vectorizer = CountVectorizer()
X_train_bow = bow_vectorizer.fit_transform(X_train)
X_test_bow = bow_vectorizer.transform(X_test)
```

fit_transform() learns the vocabulary from the training documents and converts them into count vectors. transform() converts the test documents using the already learned vocabulary. This prevents information from the test set from being used to build the vocabulary.

Step 5 — Train and evaluate the BoW classifier

```
bow_model = MultinomialNB()
bow_model.fit(X_train_bow, y_train)
bow_predictions = bow_model.predict(X_test_bow)
bow_accuracy = accuracy_score(y_test, bow_predictions)
```

MultinomialNB() creates the Naive Bayes classifier. fit() learns from the training vectors and labels. predict() assigns categories to test documents. accuracy_score() calculates the proportion of correct predictions.

Accuracy = (Number of correct predictions / Total predictions) × 100. For example, 4 correct predictions out of 5 gives 80% accuracy.

Step 6 — TF-IDF feature extraction

```
tfidf_vectorizer = TfidfVectorizer()
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)
```

TfidfVectorizer converts each document into a vector of TF-IDF weights. Words that are common across many documents are generally down-weighted, while more informative words may receive higher weights.

Step 7 — Train and evaluate the TF-IDF classifier

```
tfidf_model = MultinomialNB()
tfidf_model.fit(X_train_tfidf, y_train)
tfidf_predictions = tfidf_model.predict(X_test_tfidf)
tfidf_accuracy = accuracy_score(y_test, tfidf_predictions)
```

The same Multinomial Naive Bayes algorithm is trained using TF-IDF vectors. Its predictions and accuracy are then calculated so that they can be compared with the BoW results.

Step 8 — Predict categories for new documents

The two new sentences are transformed using the vectorizers fitted on the training set. Each trained classifier predicts a category. The expected semantic results are Technology for “Artificial intelligence helps computers learn” and Sports for “The cricket team won the tournament”. Actual predictions depend on the small training sample and learned vocabulary.

7. Expected output

The program displays the BoW vocabulary and matrix, BoW accuracy and classification report, TF-IDF vocabulary and matrix, TF-IDF accuracy and classification report, a comparison of both accuracies, and predictions for the two new documents.

```
===== NEW DOCUMENT PREDICTION =====

Document: Artificial intelligence helps computers learn
BoW Prediction: Technology
TF-IDF Prediction: Technology

Document: The cricket team won the tournament
BoW Prediction: Sports
TF-IDF Prediction: Sports
```

The exact accuracy and classification report depend on the training/testing split and the sample dataset. The displayed new-document predictions are expected outcomes, not guaranteed results.

8. How to execute the program

1. Open IDLE, VS Code, PyCharm or another Python environment.
2. Install Scikit-learn using `pip install scikit-learn`.
3. Create a Python file named **text_categorization.py**.
4. Paste the complete program and save it.
5. Open a terminal in the folder containing the file and run:

```
python text_categorization.py
```

The program requires Scikit-learn. The collections of text documents are included directly in the script, so no external dataset download is needed.

9. Difference between Bag-of-Words and TF-IDF

Bag-of-Words	TF-IDF
Counts word occurrences.	Calculates weighted word importance.
Uses frequency values.	Uses term frequency and inverse document frequency.
Common words may have high counts.	Words common across many documents are generally down-weig
Simple count-based representation.	Weighted representation based on document frequency.

10. Applications of text categorization

• Email spam detection
• News article classification
• Sentiment analysis
• Document organization
• Customer feedback classification
• Fake news detection
• Medical document classification

11. Student exercises

1. Add a third category called Politics.
2. Add five new documents under each category.
3. Change the classification algorithm from Naive Bayes to Logistic Regression.
4. Compare the accuracy of Bag-of-Words and TF-IDF.
5. Enter your own sentence and predict its category.

12. Quick knowledge check

Question 1: Which technique counts the frequency of words?
Answer: Bag-of-Words.

Question 2: What does TF-IDF measure?
Answer: Word importance based on term frequency and inverse document frequency.

Question 3: Which algorithm is used for classification in this experiment?
Answer: Multinomial Naive Bayes.

Expected laboratory outcome: Students should be able to represent text documents using Bag-of-Words and TF-IDF, train a machine-learning classifier, classify unseen documents, and compare classification performance.

Teaching suggestion: First demonstrate Bag-of-Words with a small vocabulary and a manually constructed numerical matrix. Then introduce TF-IDF and finally show how Naive Bayes uses these numerical representations to classify documents.